

CSE235 Week 12: Database with SQLite

Python is commonly used for data processing and data management. Python ships with SQLite3, a lightweight database engine that is optimized for single user access. CPD121 and CPD123 are required for most students who are taking this class, so hopefully you have some familiarity with SQL. We will first provide an overview and review of SQL and databases before exploring how it integrates with Python.

SQLite3

SQLite3 is the third major version of the SQLite database engine. SQLite stores databases as single files, making (small) SQLite databases very portable. The databases used by SQLite are relational databases that are created and queried using SQL. Let's define some terms:

- SQL - stands for "Structured Query Language". It is the dominant query language for creating, accessing, and modifying relational databases.
- Relational Database - a relational database stores data in "tables" which have schemas. The schemas are the "columns" of these tables. They are called relational databases since different tables are often related to each other via these columns.
- Primary Key - a column (or set of columns) in a database table that is used as the ID for a row. The primary key must be unique and not null (empty). Typically, an integer column that counts up from 0 is used as a primary key.
- Foreign Key - a column (or set of columns) that is used to relate one table to another
- Database - A collection of tables that are used to store data.
- Table - A collection of columns and rows that store data
- Column - A
- Row - A single data entry in a table

Creating Tables

A table is created with a `CREATE TABLE` statement. The syntax is:

```
CREATE TABLE table_name(column_list);
```

The column list consists of the column name, data type, and any additional attributes such as `NOT NULL` or `PRIMARY KEY`. Here is an example:

```
CREATE TABLE if not exists fruits(  
    id INT PRIMARY KEY AUTOINCREMENT,  
    fruit_name TEXT NOT NULL,  
    price FLOAT NOT NULL  
);
```

The first column is `id`, which has `INT` as a data type. It is a primary key, so it is automatically unique and not null. The `AUTOINCREMENT` specifies that it should automatically increase by one each time a row is added to the table. `fruit_name` is a `TEXT` column, and is specified to not be null. Attempting to set it to null or insert a row without providing a value for it will result in an error. `price` is a `FLOAT` variable that is also not allowed to be null.

Querying SQL Commands

These are the commands for querying data from existing tables.

Select

The `SELECT` statement is used to read data from a table. The syntax for a basic `SELECT` statement is as follows:

```
SELECT columns FROM table_name;
```

The `columns` can be one to many columns that exist in the table being queried. The `FROM` clause specifies the table being queried. For example, if we had the following "fruit" table:

id	fruit_name	price
----	------------	-------

id	fruit_name	price
1	Apple	1.25
2	Banana	1.05
3	Orange	1.50

If we only wanted to get the names from the table, we can run the following query:

```
SELECT fruit_name FROM fruits;
```

If we want to select all the columns, we can list them all out:

```
SELECT id, fruit_name, price FROM fruits;
```

We could also use `SELECT *`, but I don't recommend doing this. If the table is changed later on, the data returned by this query will be changed and could break client code.

Constraints

We can constrain the rows returned by a query using a `WHERE` clause. A `WHERE` clause is added after the `FROM` clause, and allows additional conditions to be added. For example, we can query all fruits that have a price less than or equal to \$1.25:

```
SELECT fruit_name FROM fruits WHERE price <= 1.25;
```

This will return:

fruit_name
Apple
Banana

A `WHERE` clause can consist of any number of conditions placed on the values of the columns.

Ordering

By default, select statements return the data from a table in the order it was inserted. If you want to sort a column, an `ORDER BY` clause can be added:

```
SELECT * FROM fruits ORDER BY price ASC;
```

This orders by ascending prices:

id	fruit_name	price
2	Banana	1.05
1	Apple	1.25
3	Orange	1.50

Alternatively, if we wanted to order by descending prices:

```
SELECT * FROM fruits ORDER BY price DESC;
```

id	fruit_name	price
3	Orange	1.50
1	Apple	1.25
2	Banana	1.05

We won't cover more complex queries like joins, since this is not a database class.

Insert

The `INSERT` statement is used to add new a row(s) to a table. The syntax for an `INSERT` statement is:

```
INSERT INTO table_name(column_list) VALUES (value_list);
```

After `INSERT INTO` is the name of the table that the new values will be

inserted into. After the table name is the list of columns that the data will be inserted into. Then, there is the `VALUES` followed by the list of values to insert.

```
INSERT INTO fruits(fruit_name, price)
VALUES ('Apple', 1.25), ('Banana', 1.05), ('Orange', 1.50);
```

Update

The `UPDATE` statement is used to update existing rows in the table. Typically, `UPDATE` statements have a where clause to indicate which row to update. Otherwise, all rows will be updated. The basic syntax is:

```
UPDATE table_name SET column=val WHERE condition;
```

And a concrete example:

```
UPDATE fruit
SET price=1.15
WHERE fruit_name = 'Banana';
```

Delete

The `DELETE` statement is used to delete rows from a table. Again, a `WHERE` clause should be used, otherwise it will delete all rows from the table. The basic syntax is:

```
DELETE FROM table_name WHERE condition;
```

Example:

```
DELETE FROM fruit WHERE fruit_name = 'Orange';
```

Python SQLite3 API

Python provides an API (abstract programming interface) for SQLite3. Like Tkinter, the SQLite3 API ships with Python. To use it, it can be imported like any of the other built-in modules:

```
import sqlite3
```

The first thing to do is to create a database connection. With SQLite, this is simple, since there is no database server/engine running in the background; the database is just a plain file. The `sqlite3.connect()` function is passed the path to a database file and creates a connection to it.

```
import sqlite3

# if the database file does not exist, it will be created
connection = sqlite3.connect("fruit.db")
```

The next thing to do is to create a cursor object to execute queries against the database. The `sqlite3.cursor()` function is passed a connection as an argument, and returns a new cursor:

```
import sqlite3

connection = sqlite3.connect("fruit.db")
cursor = sqlite3.cursor(connection)
```

The cursor object has the method `execute()` associated with it. This can be used to execute SQL commands. For example, we can use the cursor to create a table:

```
import sqlite3

connection = sqlite3.connect("fruit.db")
cursor = sqlite3.cursor(connection)

# executing a CREATE TABLE command
cursor.execute("""
CREATE TABLE if not exists fruits(
    id INT PRIMARY KEY AUTOINCREMENT,
    fruit_name TEXT NOT NULL,
    price FLOAT NOT NULL
);
""")

# inserting values into the table
cursor.execute("""
INSERT INTO fruits(fruit_name, price)
```

```
VALUES ('Apple', 1.25), ('Banana', 1.05), ('Orange', 1.50);
""")
```

This will create the fruit table and add the data from the examples. The cursor can also be used to execute select queries. A list of lists is returned:

```
# execute the select query
cursor.execute("SELECT * FROM fruits;")
# fetch all the rows selected
rows = cursor.fetchall()
# iterate through the rows
for row in rows:
    print(f"ID: {row[0]}")
    print(f"Name: {row[1]}")
    print(f"Price: ${row[2]}")
```

The delete and update queries can be used similarly. But, what if we want to make the queries interactive? The Python SQLite3 API allows us to use question marks as placeholders in queries. We then pass a tuple of values to replace the question marks with. For example, the following code snippet will read values from the user to insert a new row into the fruit table:

```
name = input("Enter Fruit Name: ")
price = float(input("Enter Fruit Price: $"))
cursor.execute(
    """INSERT INTO fruits(fruit_name, price)
VALUES(?,?)
""", (name, price)
)
```

The value of `name` will replace the first question mark, and the value of `price` will replace the second question mark. This can be used for any type of query.

Conclusion

This week we discussed the SQLite3 API for Python. The API is simple, using it primarily consists of using the `connect()` function to open a database file, and the `cursor()` function to create a cursor to execute queries against the database. An larger SQLite3 example is included

with this week's example. It features a Tkinter GUI to interact with a database of pets.

Documentation built with [MkDocs](#).